WEEKS 1–2 MADE TEXT INTO NUMBERS

Numbers become predictions

Your ISM 6251 toolkit meets 12,959 sparse columns — and we find out, honestly, which models survive the trip.

TODAY · 12:30 – 3:30

Six blocks, one honest question each

12:40

Quiz 2
Honorlock · locks 12:55

12:55 · 01

The honest pipeline
Fit on train only — or lie

1:20 · 02

Linear & kNN
The baseline to beat

1:50 · 03

Trees, and SVD
Why your best models lose

2:30 · 04

Neural nets, gently
A network from parts

3:00 · 05

Evaluation, honestly
When averages lie

3:18 · 06

Choosing & the ceiling
What CV cannot see

3:27

Wrap-up
Assignment 3 · the router

Route Northwind's inbox — 1,000 tickets a day, five queues

- Dispatchers are **eight hours behind**; a mis-route costs **about a day**.
- You hold **3,000 routed tickets**. This morning's 400 are unrouted — routing them **is** the job.

> "What I can't have is **confidently wrong**."
>
> — Maya Chen, support operations



One ticket becomes one row of 12,959 numbers

RAW TEXT

One ticket

“Order #8841 arrived damaged, requesting a refund…”

→

TOKENIZE · W1

Words

sklearn’s default: lowercase, tokens of 2+ letters (\w\w+), punctuation dropped →

`order · damaged · refund`

→

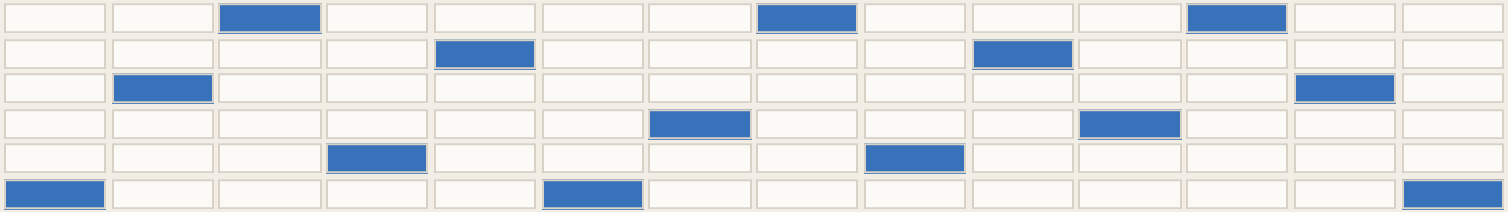
WEIGH · W2

TF-IDF row

`[ 0  .42  0  .17  0 … ]`

design matrix X — one column per word that survives the tokenizer + stop_words + min_df → 12,959

99.5% zeros



y

- billing
- shipping
- returns
- product
- account
- billing

NEW THIS WEEK

◆ Attach a label column and fit a classifier — that is the whole change. The sparsity you see above decides everything that follows.

Anything with a `.fit()` learns from train only

WRONG

1
Fit on ALL rows
The vectorizer studies every ticket.

→

2
Then split
Held-out rows already shaped the features.

→

RESULT
Inflated, quietly
Looks great. Means nothing.

RIGHT

1
Split FIRST
Held-out rows go behind glass.

→

2
Fit inside the Pipeline
`Pipeline(Tfidf, LogReg)` — each fold refits on its own rows.

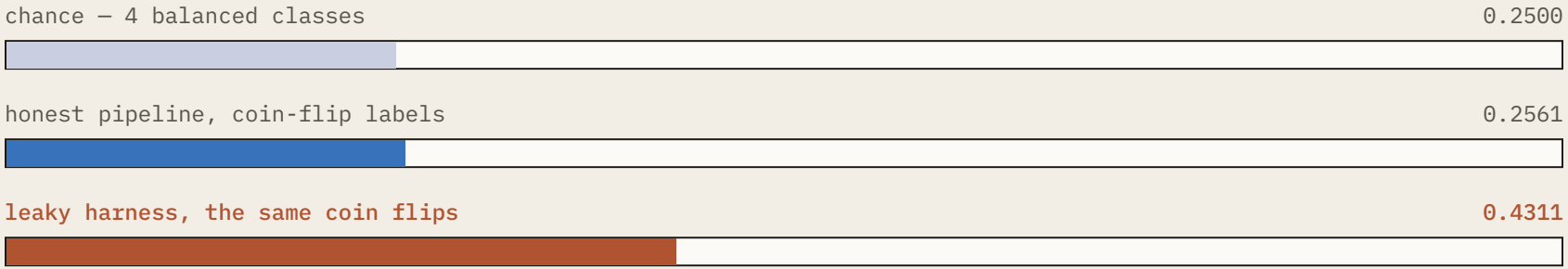
→

RESULT
Honest estimate
Predicts deployment, not memory.

◆ Vectorizers, scalers, selectors, SVD — even a tokenizer you train. If it saw a row it is later scored on, the number is wrong.

On coin-flip labels, the leaky harness still “scores” 0.43

Pick the “300 most predictive words” using **all** the labels, then split. Now replace every label with a coin flip — and watch.



▲ **+18 points that do not exist** — no signal, so the leak is the only thing measured.

◆ The one-line diagnostic: rerun your harness on **shuffled labels**. If it beats chance, the harness is broken.

Three splits, three jobs



THE MODEL LEARNS HERE

Train

Every .fit() draws its numbers from these rows only.

DECISIONS HAPPEN HERE

Validation

Compare, tune, stop. Consulted many times — its score drifts optimistic.

TOUCHED ONCE

Test

One run at the end: the honest number for the memo.

◆ Every extra peek at the test set converts it into a second validation set. Report the best of eight tries and you have reported the luckiest coin.

Meet the task: 12,959 numbers in, one label out

IN

One ticket, as numbers

`[ 0 .42 0 .17 0 … ]` — 12,959 values (the width the tokenizer set), mostly zero

→

?

A classifier

Some machine that turns the row into one score per class.

→

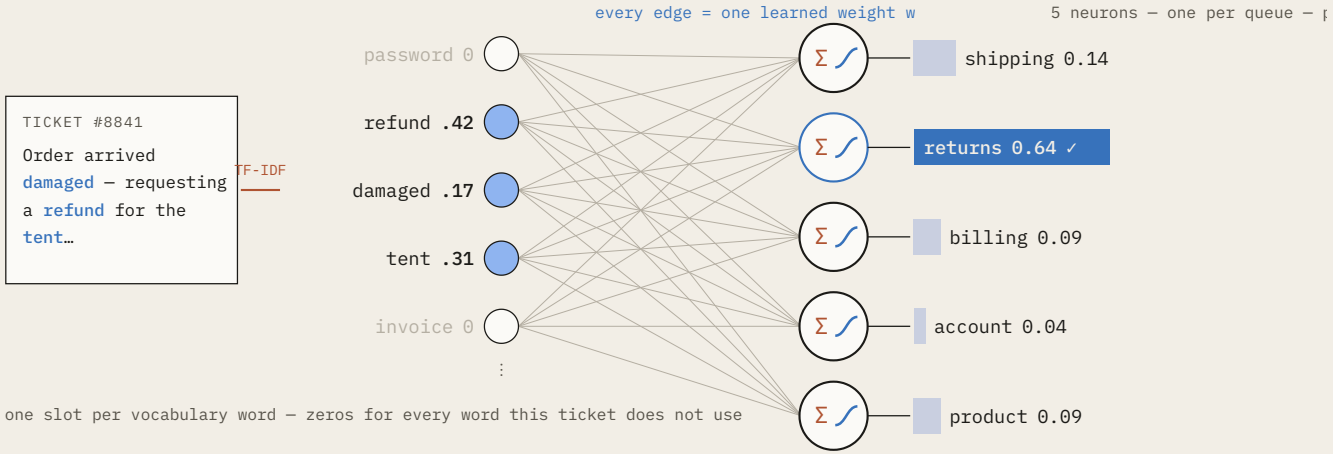
OUT

A label — and a probability

The router needs a confidence it can threshold, not just a name.

- Whatever the model — linear, tree, network — the job is identical: **one score per class, from the row's numbers**. Biggest score wins.
- Strategy for this block: start with the **simplest scorer you already own** from ISM 6251…
- …and make every fancier model **earn its keep against it**.

Logistic regression is a one-layer network



$score_q = \Sigma\ x_j \cdot w_{q,j} + b_q$ → σ squashes to 0–1; softmax: the queues sum to 1 → biggest wins: route to returns

◆ Parameters = vocabulary × classes + classes. Measured on the 4-topic benchmark: 12,959 × 4 + 4 = **51,840 numbers, scoring 0.8269** — the bar. Block 4 stacks a second layer onto this exact drawing.

Read the coefficients — sometimes they confess

The five-queue model you just saw drawn, opened up: sort the **billing** neuron's weights and read the top of the list. Measured on our ticket training split — lecture notebook §4.5.

top of the billing list	coef
charged	2.77
card	2.28
billing	2.22
but	**2.09**
price	1.85
99	**1.84**

- → The top three are **exactly what you would hope**: charged, card, billing. The audit reassures…
- → …until **"but"** — with "was" and "not" just below — billing is where customers **argue**: complaint grammar as evidence. And **"99"**: the tails of prices.
- → Fragile: **any** dispute-shaped ticket pulls toward billing, whatever it is about.

◆ Reading the weights is a deployment audit, not a nicety — ten minutes caught a risk no accuracy number shows.

Not every model gives you probabilities

Your router thresholds a confidence. **LinearSVC** optimizes margins — it has no **predict_proba** to threshold, and the first request crashes.

model	accuracy	probabilities?
`LinearSVC(C=1)`	`0.8209`	`No — AttributeError`
`LogisticRegression(C=10)`	**`0.8269`**	`Yes, natively`

→ For a router the choice makes itself: **logistic regression** has real probabilities built in — and gives up nothing here.

⚠ “But the SVC *does* give a number — its distance from the boundary?” It does — and **a distance is not a probability**: arbitrary units, unbounded, nothing sums to 1. Being 1.9 units from the line is not being 70% sure.

kNN: the distance function *is* the model

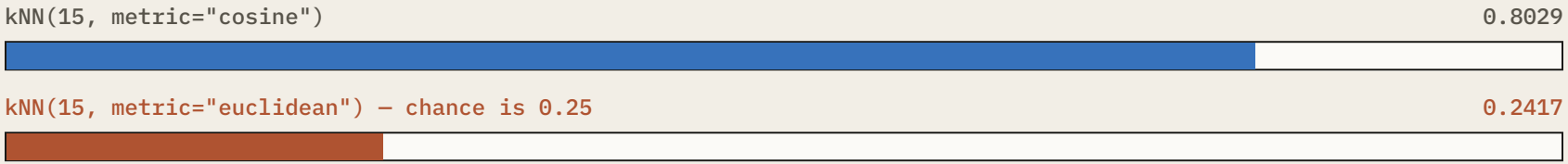
query — short refund ticket: refund · damaged

A — same topic, 3× longer: refund · damaged · refund · return · damaged …

B — different topic, similar length: password · locked

Cosine compares direction — the hue — and picks **A**. ✓

Euclidean is dominated by length — and picks **B**. ✗



Random forest and XGBoost lose on sparse text

A TREE ASKS ONE WORD AT A TIME

```
refund > 0.12 ? no
  └ tracking > 0.08 ? no
    └ invoice > 0.05 ? no…
```

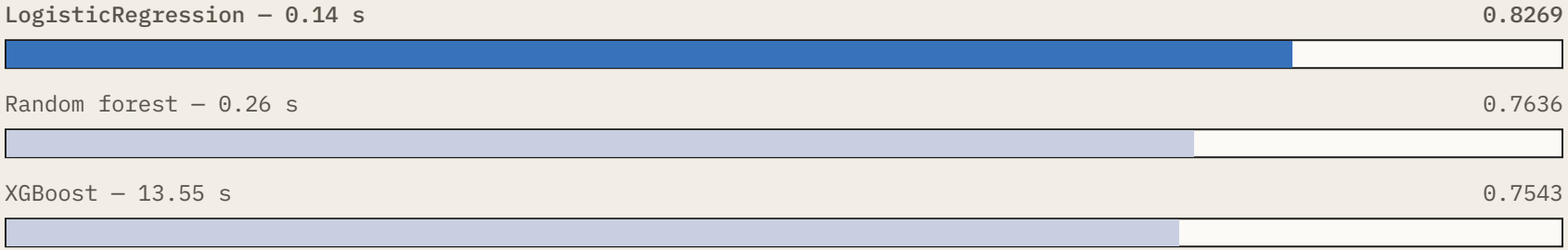
Each split sees one column of 12,959 — and almost every cell is zero.

THE SUM READS EVERY CLUE AT ONCE

refund, a little →
damaged, a little →
tent, barely →
arrived, barely →
…a hundred more →

Σ

a confident call



Change the shape of the data: TruncatedSVD

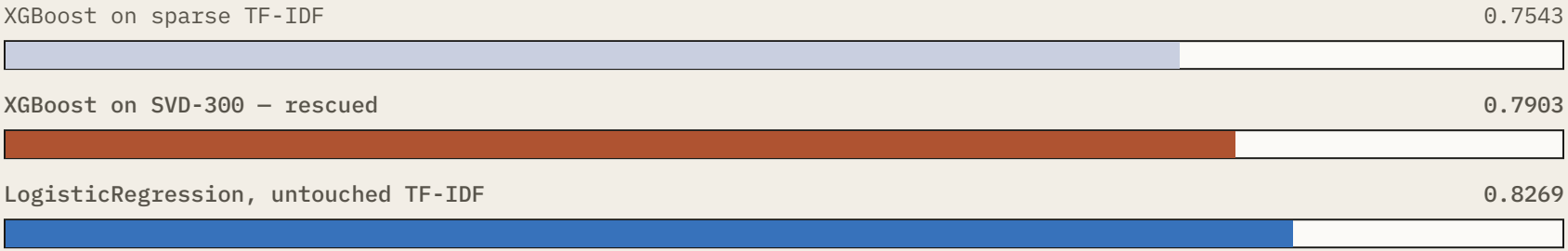
SPARSE · WIDE
12,959 word columns
Almost every cell zero. Trees starve.

→

LOSSY COMPRESSION
TruncatedSVD(200)
Blends co-occurring terms. No labels; nothing added.

→

DENSE · NARROW
200 columns
92.5% of variance kept — small enough to feed a small network.



◆ Match the model to the shape of the data — or change the shape. The same SVD powers the neural network after the break.

The leaderboard at the break

model	test acc	macro F1	probabilities?
LogisticRegression(C=10)	**0.8269**	**0.8177**	**Yes, natively**
LinearSVC(C=1)	0.8209	0.8111	No — AttributeError
kNN(15, cosine)	0.8029	0.7969	Vote fractions — coarse
XGBoost on SVD-300	0.7903	—	Yes
Random forest (sparse)	0.7636	0.7481	Vote fractions
XGBoost (sparse)	0.7543	0.7446	Yes
kNN(15, euclidean)	**0.2417**	**0.2137**	**Yes, and meaningless**

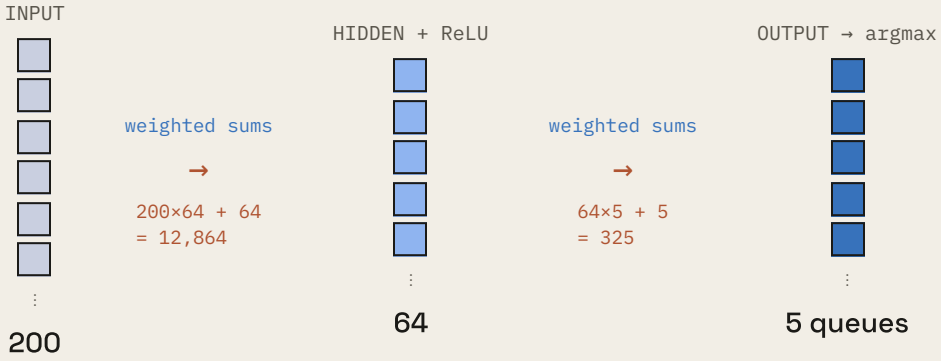
◆ Three lessons, not a ranking: linear wins on sparse text · the metric is part of the model · prep moves trees more than tuning does.

BLOCK 4 / 6 · 2:30

Neural networks, gently

Weighted sums plus a kink, trained by a five-line loop. One genuinely new idea — the rest is assembly.

Two layers, 13,189 numbers — all of them countable



- Each layer is logistic regression's engine, stacked. **13,189 parameters** — 6× smaller than on raw TF-IDF; the SVD prep bought that.

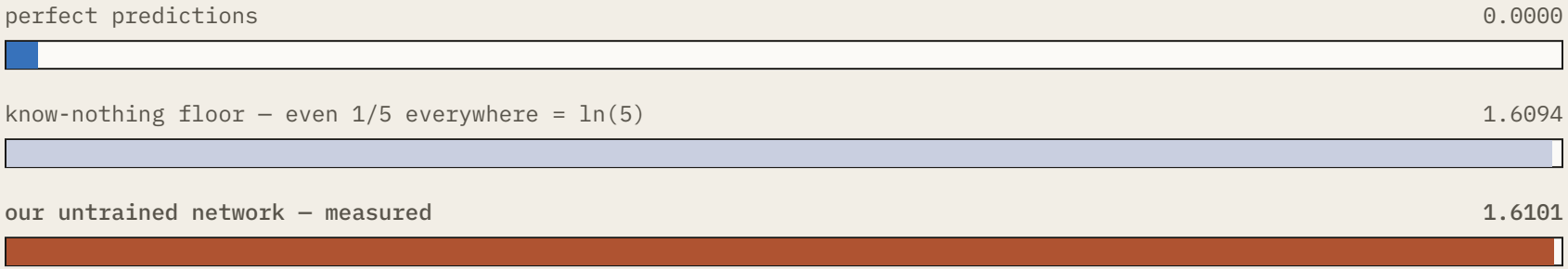
ReLU: replace negatives with zero. That is the entire secret.

Linear → Linear = one Linear — sums of sums are still sums. Nothing gained.

Linear → ReLU → Linear ≠ — the kink stops the collapse. Layers now earn their keep.

in → ReLU → −2.0 → **0** | −0.5 → **0** | 0.5 → **0.5** | 2.0 → **2.0**

The loss is one number — and it starts at “know nothing”



← Training = push this number down. Watching it fall **is** watching learning happen.

Autograd: one call, a slope for all 13,189 weights

SANITY CHECK — ONE NUMBER

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
x.grad → 6.0
```

The slope of x^2 at 3 is 6 — and no calculus was typed.

- → **loss.backward()** — the same trick on the real loss: a slope for every weight, one call.
- → To lower the loss, move each weight **against** its slope.
- → **Adam** does the moving — a small step, every weight, every batch. Step size: lr = 0.001.

◆ That is the whole mechanism. Everything else in deep learning is assembly around these three answers.

The training loop: five lines, repeated

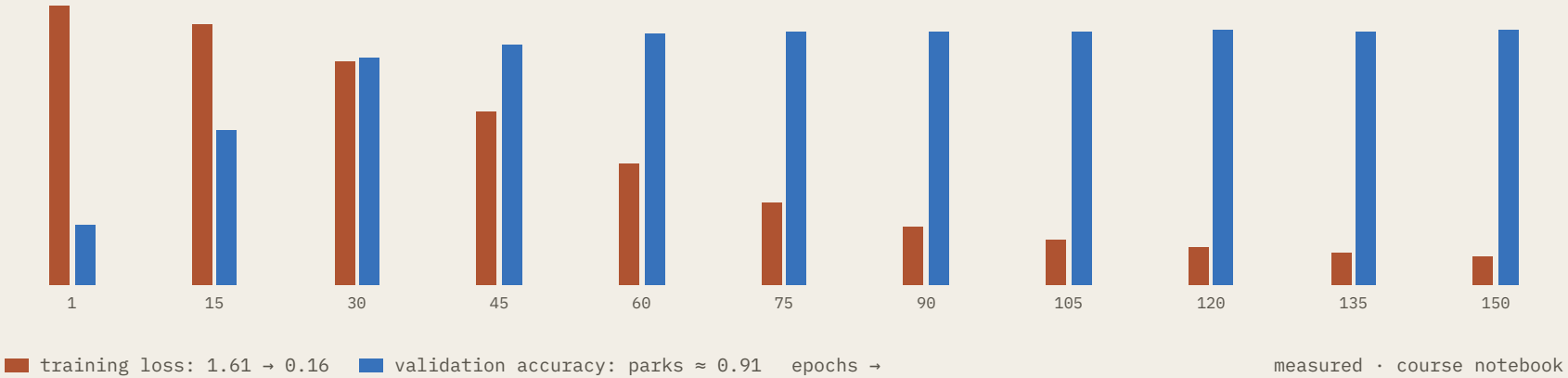
1	`optimizer.zero_grad()`	clear old slopes — they accumulate
2	`predictions = model(batch)`	forward: 200 tickets through the layers
3	`loss = loss_fn(predictions, y)`	one number: how wrong
4	`loss.backward()`	autograd: every slope, one call
5	`optimizer.step()`	nudge every weight against its slope

↻ 9 batches of 200 = **one epoch**

✓ each epoch: score **validation**, gradients off — next slide

MEASURED, NOT ASSERTED

Loss falls forever. Validation tells you when to stop.



◆ Remember the best weights · stop after **patience** = 10 quiet epochs · roll back. That is **early_stopping=True**, built from parts you can see.

The whole road, end to end — what did capacity buy?



same splits · each tested once	params	test acc	macro F1
logistic regression	3,185	0.8857	0.8772
network, raw TF-IDF (636→128→5)	82,181	0.8959	0.8879
network, SVD-200 (200→64→5)	**13,189**	**0.8976**	**0.8906**

- Capacity bought **about one point** — 82,181 parameters over 3,185. At this scale, the model is not the story.
- SVD — Block 3's tool, new job — shrinks the **input** so the network shrinks: **6× fewer parameters, nothing lost**.
- What you keep is **the road**: honest splits, train-only fits, validate to stop, test once.

◆ Early stop at epoch 31 · validation 0.9181 · each test number spent once. Week 4 swaps only the model inside this road — lecture notebook §4.4 walks the SVD run; the step-by-step notebook builds the raw-TF-IDF run.

One score is not the truth: three honest questions

The bake-off crowned a number. Before anyone ships it, three questions a single average cannot answer — one per slide.

01 · THE CONFUSION MATRIX

Which mistakes, exactly?

An average hides who pays for the errors.

02 · CALIBRATION

Do the probabilities mean anything?

Your router's threshold consumes them either way.

03 · READ THE ERRORS

What do they have in common?

Some defects only appear when you read the documents.

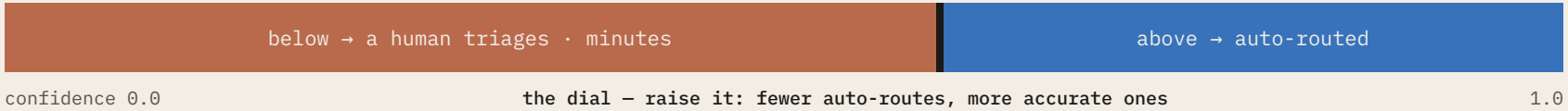
The confusion matrix names the failure

rows = true queue · columns = predicted · n = 586

	shipping	returns	billing	account	product
shipping	**161**	6	3	1	2
returns	2	**117**	1	3	1
billing	3	4	**104**	4	1
account	0	3	2	**62**	2
product	4	3	1	4	**92**

- Diagonal = correct: 536 of 586 → accuracy **0.9147**.
- Biggest error cell: **6 shipping tickets sent to returns** — and 2 back the other way. Shipping↔returns is **the confusable pair**.
- Those are **the two teams who notice your router first** — name the cell out loud: “true X, predicted Y”.

Can you trust the probabilities? The threshold has to.



"0.65" → 94%

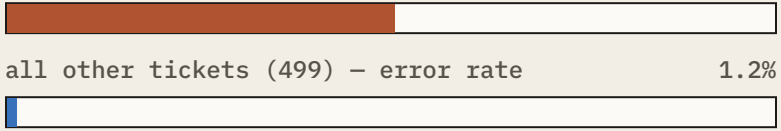
Measured on our validation split: when this model's confidence reads **0.6–0.7** (71 tickets), it is right **94%** of the time — *under*-confident. "Auto-route above 0.70" would send those 71 tickets — 67 of them **correctly routed** — to a human for no reason.

◆ If a threshold consumes a probability, check the probability first — on your validation set, before any dial is chosen. Assignment Task 6: build the coverage–accuracy table, then pick.

15% of the tickets cause 88% of the errors

"My tent arrived **damaged** — also, was I **charged twice**?"

one ticket, two intents — the label follows the *first*; the model reads the whole bag



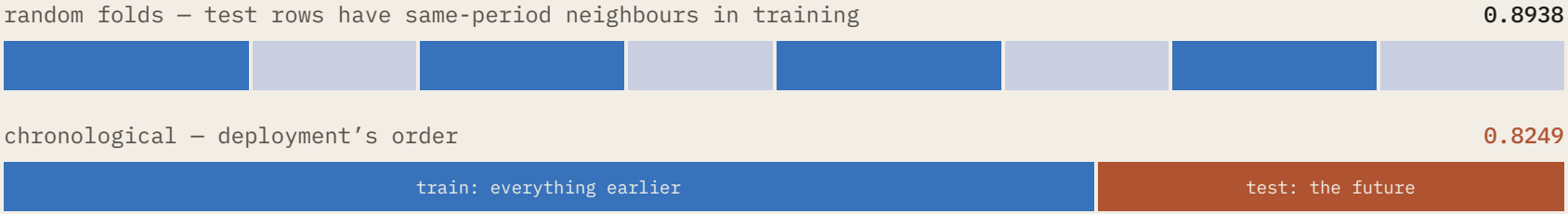
- → Validation makes **50 errors**; the two-intent cohort carries **44 of them** — 88%.
- → Also measured: the model **knows** it is confused — average confidence **0.61** on two-intent tickets vs **0.79** elsewhere…
- → …so the **threshold dial** from the last slide catches many of them. Handled, not hidden.

◆ No aggregate metric names this cohort — reading twenty misrouted tickets does. Assignment Task 5: quantify it on the full corpus; Assignment Task 6's threshold is one honest answer to it.

MEASURED ON A DATED CORPUS

Cross-validation cannot see the future

Our tickets carry no dates, so this lesson is measured on the dated benchmark corpus — and it applies to every system you will ever deploy.



▼ **6.9 points, none of it cheating**: threads died, terminology shifted, new posters arrived. Your inbox drifts the same way — a sale, a new product line.

◆ Time index? Hold out the **most recent** slice, not a random one — and plan for the deployed score to decay.

The ceiling: opposite meanings, one identical vector

"the repair was **not good** and the refund was **fast**"

"the repair was **good** and the refund was **not fast**"

```
[1 1 1 1 1 1 2 2] = [1 1 1 1 1 1 2 2]        identical rows? True
```

- Order was destroyed at encoding time — no model downstream can recover it. Week 4: models read words **in order**. Your loss, optimizer, and loop carry over unchanged.

THIS WEEK

DUE TUESDAY, SEP 15 · 11:59 PM

Assignment 3 — the ticket router

Route the 400-ticket batch, pick your threshold, write Maya's memo. **Practice assignment first** — it rehearses every graded mechanic.

NEXT WEDNESDAY · 12:40

Quiz 3 — this material

Honorlock, closed everything. Practice quizzes 3A & 3B close Tuesday night. Then Week 4: models finally read **order**.

The PyTorch notebooks — `pytorch-intro` · `pytorch-step-by-step` — pair with the posted video.